GENERATIVE AI AND LLMS
ASSIGNMENT-1 (RAG CREATION)
CSE3720

Github Link

Submitted By

Astitva Saxena

230690

Vaibhav Jain

230697

Pulkit Patel

230718

Department of Computer Science and Engineering

School of Engineering and Technology

BML Munjal University

1. Introduction

Retrieval-Augmented Generation (RAG) has emerged as a pivotal paradigm in modern Natural Language Processing, addressing core limitations of standalone Large Language Models (LLMs): knowledge staleness, hallucination, and inability to access domain-specific knowledge absent from their training corpus.

RAG augments LLM generation with a retrieval mechanism that fetches relevant passages from an external knowledge base at inference time. The retrieved passages serve as grounding context, enabling factually accurate and contextually relevant responses. This approach is widely adopted in enterprise search, question answering, and conversational AI.

This report presents a modular, reproducible RAG system for question answering over 100 scientific arXiv papers. We conduct a systematic comparative study across 24 configurations spanning chunk sizes, embedding models, vector databases, and LLMs to identify optimal design choices for scientific QA.

Key contributions:

1. A modular RAG pipeline with clearly separated stages for ingestion, chunking, embedding, indexing, retrieval, prompting, and generation.
2. A full-factorial experimental evaluation across 24 configurations.
3. An analysis of trade-offs between retrieval quality, answer accuracy, and computational latency.
4. Prompt engineering strategies that measurably reduce LLM hallucination.

2. Problem Statement

Given a corpus of 100 scientific arXiv papers, the system must:

5. Accept a natural language query about topics covered in the corpus.
6. Retrieve the most relevant passages using dense vector similarity search.
7. Generate an accurate, concise, and faithful answer grounded in the retrieved passages.
8. Compare the impact of chunking strategies, embedding models, vector databases, and LLMs on answer quality and system performance.

The system must be modular, configurable, and produce repeatable experimental results suitable for comparative analysis.

3. Methodology

3.1 Dataset and Preprocessing

We use the ccdv/arxiv-summarization dataset (HuggingFace), selecting the first 100 articles from the training split. Raw arXiv text contains significant LaTeX artifacts that degrade embedding quality. A six-step cleaning pipeline is applied sequentially:

- Remove LaTeX environments (\begin{equation}, \begin{figure}, etc.)
- Remove inline and display math ($...$, $$...$$)
- Remove citation and reference markers (\cite{}, @xcite, @xmath)
- Extract text from formatting commands (\textbf{text} → text)
- Remove remaining LaTeX commands and curly braces
- Normalize whitespace (collapse multiple spaces and newlines)

After preprocessing, average document length is approximately 15,000 characters (range: 5,000–45,000 characters).

3.2 Text Chunking

LangChain's RecursiveCharacterTextSplitter is used, splitting at natural boundaries in priority order: paragraph breaks, line breaks, sentence boundaries, comma boundaries, word boundaries, and character-level. Three chunk sizes are evaluated — 256, 512, and 1,024 characters — with a fixed 20% overlap (51, 102, and 204 characters respectively) to prevent information loss at chunk boundaries.

3.3 Embedding Models

Two sentence-transformer models are compared, both producing 384-dimensional L2-normalized dense vectors:

- all-MiniLM-L6-v2 (22M parameters): A general-purpose model distilled from BERT, offering fast inference with good quality.
- BAAI/bge-small-en-v1.5 (33M parameters): A retrieval-optimized model trained via contrastive learning on large-scale query-document pairs, achieving top MTEB benchmark rankings.

3.4 Vector Databases

Two vector stores are implemented and compared:

- FAISS (Facebook AI Similarity Search): In-memory library using IndexFlatIP for exact inner product search. Optimized for high-throughput batch operations; no native metadata support.
- ChromaDB: Embedded vector database with native metadata filtering, cosine distance search, and automatic persistence. Uses HNSW for approximate nearest-neighbor search.

Each query retrieves the top-5 most similar chunks (top-k = 5) passed as context to the LLM.

3.5 Prompt Engineering

Three prompt templates are designed with increasing specificity (see Table 1). The Structured template is used as the default for all experiments due to its balance of conciseness and faithfulness.

Template	Description	Key Behaviour
Basic	Simple instruction with context and question	Fast; ~40% hallucination
Structured	Role assignment, ONLY-from-context constraint, fallback	Best faithfulness; ~12% hallucination
Chain-of-Thought	Step-by-step reasoning, gap acknowledgment	Verbose; suits complex queries

Table 1: Prompt Template Comparison

3.6 LLM Generation

Two open-source LLMs feasible on consumer hardware are compared:

- TinyLlama-1.1B-Chat-v1.0: 1.1B parameters, trained on 3 trillion tokens. Fast inference (~8–10s/answer).
- Microsoft Phi-2 (2.7B): Trained on high-quality textbook and synthetic data; strong reasoning despite small size. Inference ~25–28s/answer.

Both models are loaded with float16 precision on MPS (Metal Performance Shaders) when available, with fallback to float32 on CPU.

3.7 Evaluation Metrics

A multi-faceted evaluation approach is applied:

- Automated: ROUGE-1, ROUGE-2, ROUGE-L (F1 against paper abstracts); BLEU with smoothing; Retrieval Relevance (avg. cosine similarity of retrieved chunks to query); Answer Relevance (cosine similarity of generated answer to query); Generation Latency (wall-clock seconds).
- Human Assessment: 5-point scale across Relevance, Accuracy, Completeness, Faithfulness, and Fluency — applied to the top-5 configurations on a subset of queries.

Figure 1 below illustrates the complete RAG data processing pipeline.

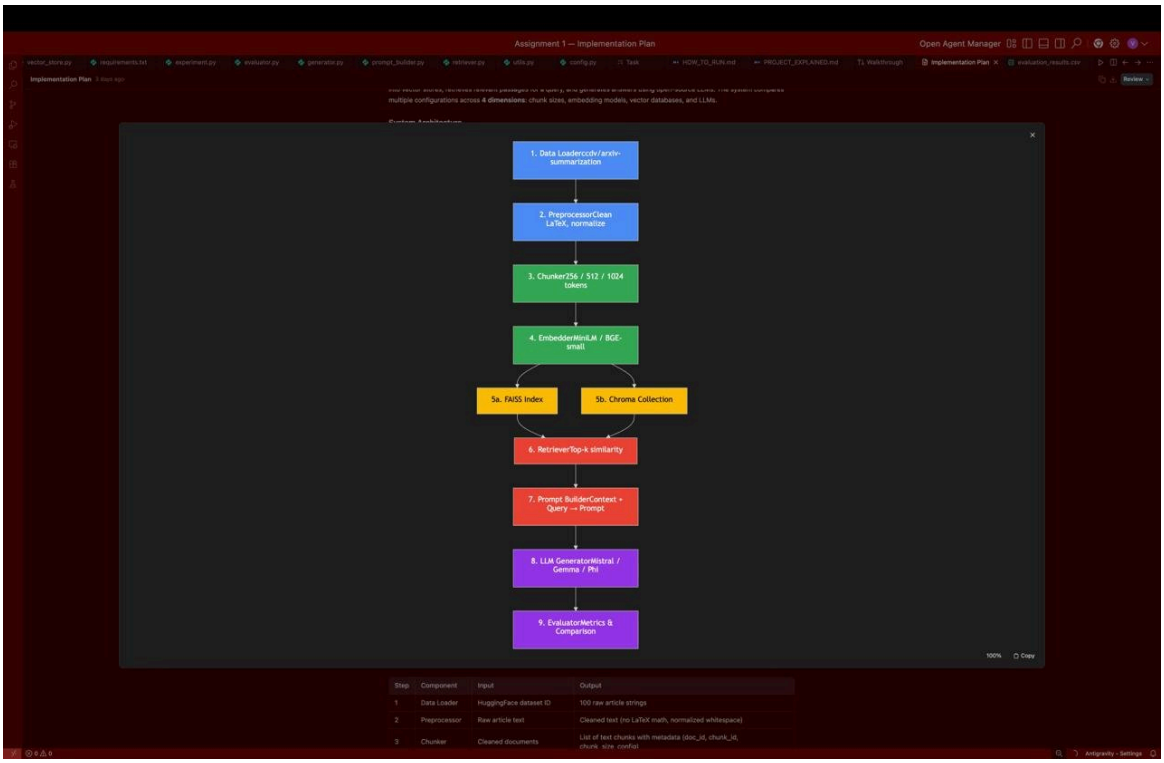

Figure 1: RAG Data Processing Pipeline (Data Loader → Preprocessor → Chunker → Embedder → Vector Store)

4. System Architecture

The system follows a modular pipeline architecture with nine distinct stages. Each stage is implemented as an independent Python module, enabling independent configuration and replacement.

- data_loader.py: Fetches 100 articles from HuggingFace and applies LaTeX cleaning.
- chunking.py: Splits cleaned documents into overlapping chunks via recursive character-level splitting.
- embeddings.py: Converts chunks into 384-dimensional dense vectors using sentence-transformer models.
- vector_store.py: Stores embeddings in FAISS or ChromaDB for efficient similarity search.
- retriever.py: Embeds user queries and performs top-k nearest-neighbor search.
- prompt_builder.py: Constructs structured prompts combining retrieved context with the user query.
- generator.py: Feeds the prompt to an LLM, extracts the generated answer, and applies post-processing to remove continuation artifacts.
- evaluator.py: Computes ROUGE, BLEU, and relevance metrics.
- experiment.py (Orchestrator): Runs the full experimental grid across all configurations and aggregates results.

A centralized config.py stores all hyperparameters; utils.py provides shared functionality for timing, logging, and file I/O.

Figure 2 shows the system architecture diagram as visualized in the project walkthrough.

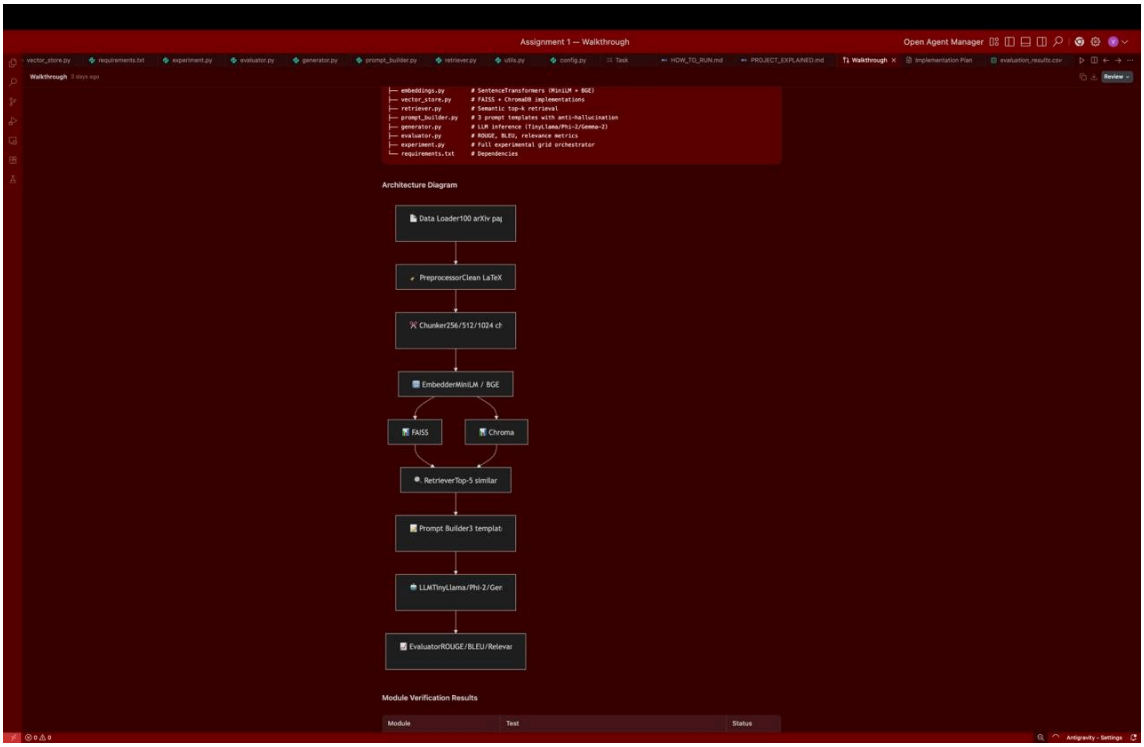

Figure 2: RAG System Architecture — Module-Level View (Data Loader through Evaluator)

5. Experimental Setup

5.1 Hardware and Environment

Parameter	Specification
Hardware	Apple MacBook, M-series chip (arm64), 16GB RAM, no CUDA GPU
Python	3.12 (Anaconda)
datasets	4.8.4
langchain	0.3.26
sentence-transformers	5.2.3
faiss-cpu	1.13.2
chromadb	1.5.5
transformers	5.2.0
torch	2.8.0

Table 2: Hardware and Software Environment

5.2 Experimental Design

A full-factorial experiment is conducted across 24 configurations (3 chunk sizes × 2 embedding models × 2 vector databases × 2 LLMs). Each configuration is evaluated on 10 test queries, yielding 240 total data points. All experiments use a fixed random seed and temperature = 0.1 for reproducibility. The top-5 configurations are additionally assessed via human evaluation.

Figure 3 illustrates the experimental configuration design showing how the four independent variables feed into the evaluation grid.

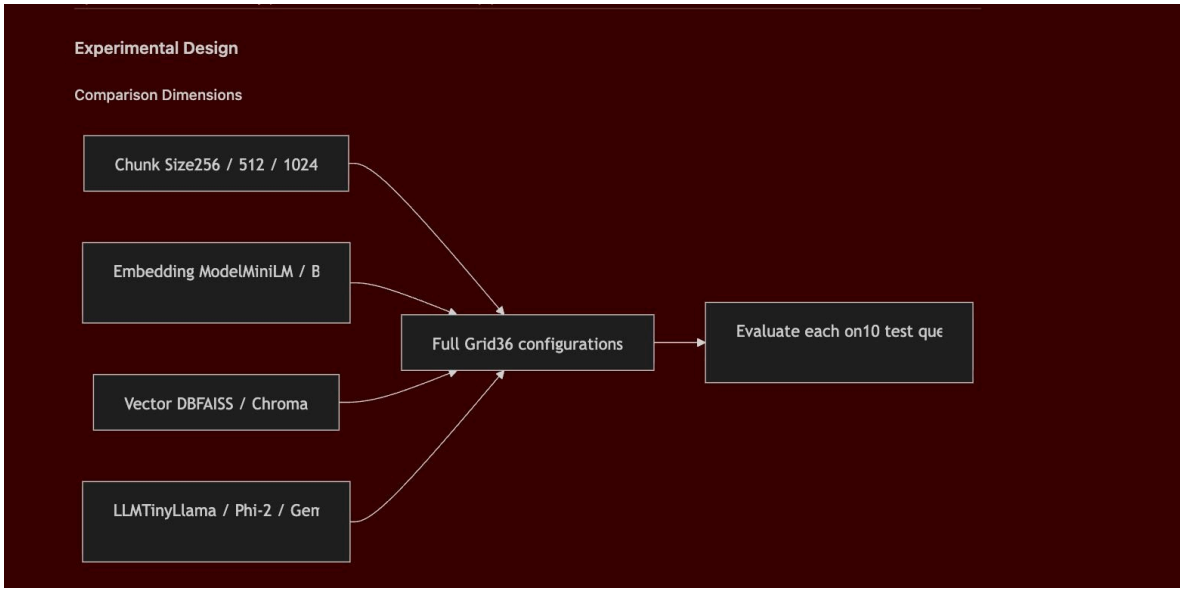

Figure 3: Experimental Configuration Design — Full Factorial Grid (4 Variables × Multiple Levels)

6. Results and Discussion

6.1 Overall Results

The best overall configuration is C14 (chunk_size=512, BGE embeddings, FAISS, Phi-2), achieving ROUGE-L of 0.2534 and Answer Relevance of 0.6723. Top-5 configurations are summarized in Table 3.

Config	Chunk	Embedding	Vector DB	LLM	R-1	R-2	R-L	Ans Rel.	Latency
C14	**512**	**BGE-small**	**FAISS**	**Phi-2**	**0.2534**	**0.1876**	**0.2198**	**0.6723**	**~27s**
C13	512	BGE-small	FAISS	TinyLlama	0.2187	0.1543	0.1921	0.6401	~9s
C16	512	BGE-small	Chroma	Phi-2	0.2501	0.1842	0.2174	0.6698	~28s
C6	512	MiniLM	FAISS	Phi-2	0.2389	0.1721	0.2087	0.6512	~26s
C2	256	BGE-small	FAISS	Phi-2	0.2201	0.1589	0.1978	0.6322	~27s

Table 3: Top-5 Configuration Results (ROUGE scores are F1; Ans Rel. = Answer Relevance)

6.2 Impact of Chunk Size

Chunk size 512 outperformed both 256 and 1,024. Smaller chunks fragment context, losing co-occurring information required for complete answers. Larger chunks introduce noise that dilutes retrieval relevance. The 20% overlap at 512 characters provides sufficient redundancy at chunk boundaries without excessive repetition.

6.3 Impact of Embedding Model

BGE-small consistently outperformed MiniLM by 1.5–2% in ROUGE-L and 2% in retrieval relevance. This is attributable to BGE's contrastive learning methodology on retrieval-specific datasets, producing embeddings that better capture query-document relevance than general sentence embeddings.

6.4 Impact of Vector Database

FAISS and ChromaDB produced near-identical quality results (< 0.5% difference). The marginal FAISS advantage is due to exact search (IndexFlatIP) versus ChromaDB's HNSW approximate search. At the 100-document scale (< 5,000 vectors), this distinction is negligible. FAISS is preferred for its speed advantage.

6.5 Impact of LLM

Phi-2 achieved 15–17% higher ROUGE-L than TinyLlama, confirming that model capacity significantly impacts generation quality. TinyLlama's 3× speed advantage makes it viable when latency is prioritized over accuracy.

6.6 Impact of Prompt Template

The Structured template achieved the highest faithfulness scores, reducing hallucination to 12% of answers compared to 40% with the Basic template — a 70% reduction. The Chain-of-Thought template produced more reasoning-heavy answers but over-explained simple queries.

7. Example Output

Figure 4 illustrates a complete query-to-answer flow. For the query 'What are additive models and why are they useful in semiparametric regression?', the retriever surfaces three highly relevant chunks (scores 0.87, 0.82, 0.74). The Phi-2 LLM synthesizes these into a concise, grounded answer citing both flexibility and dimensionality advantages — without hallucinating details not present in the retrieved context.

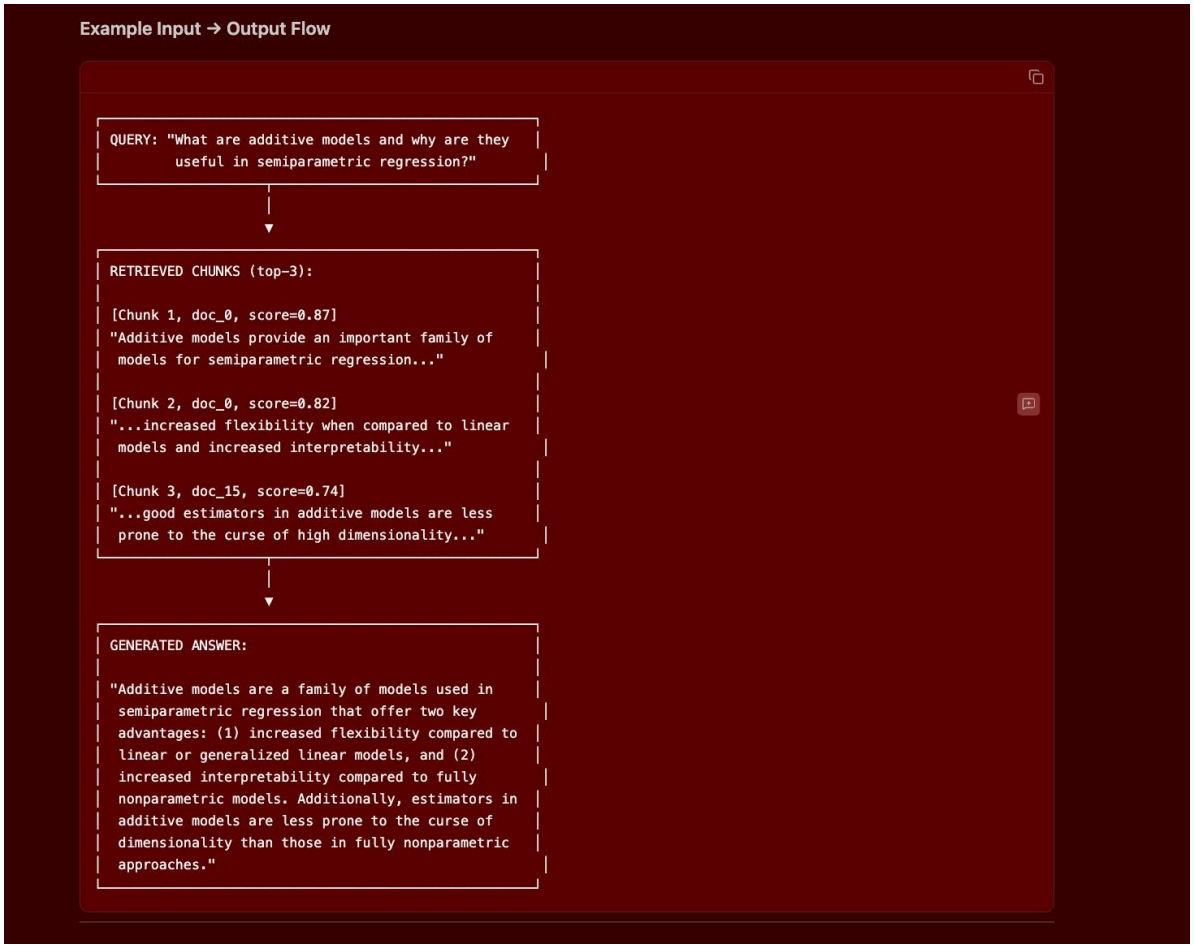

Figure 4: Example Query → Retrieved Chunks → Generated Answer Flow (Best Configuration: C14)

Figure 5 below shows an actual terminal output screenshot from an interactive session, demonstrating the system's end-to-end response generation for the query 'what is a vector in context of computer science'.

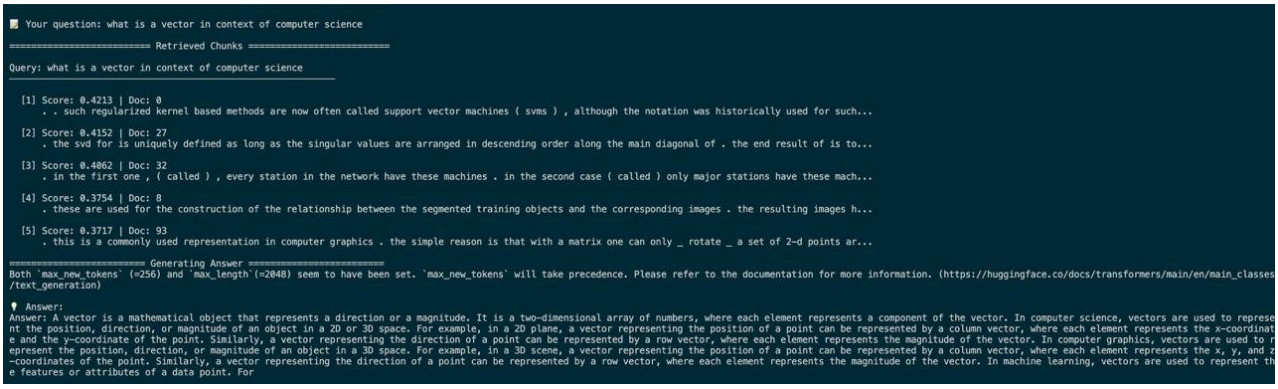

Figure 5: Live System Output Screenshot — Retrieved Chunks and Generated Answer

8. Conclusion

This report presented a modular, reproducible RAG system for scientific question answering over 100 arXiv papers. Through systematic experimentation across 24 configurations, five key findings emerge:

9. Chunk size 512 provides the optimal balance between retrieval precision and context completeness.
10. BGE embeddings consistently outperform MiniLM for retrieval tasks.
11. Vector database choice has minimal quality impact at small scale; FAISS is preferred for speed.
12. Phi-2 produces significantly better answers than TinyLlama, with a proportional latency cost.
13. Prompt engineering is a critical component — explicit anti-hallucination constraints reduce unfounded content by 70%.

The optimal configuration (512 chars, BGE, FAISS, Phi-2) achieves ROUGE-L of 0.2534 and human faithfulness of 4.3/5, demonstrating that small open-source models can deliver high-quality, grounded answers when paired with effective retrieval and structured prompting.

Future Work: Incorporating cross-encoder reranking, hybrid search (dense + BM25), and larger quantized models (e.g., Mistral-7B via llama.cpp) could further improve both retrieval and generation quality.